



Instituto Tecnológico de Buenos Aires

LA RECETA

Métodos Numéricos

Índice general

1. Ecuaciones No Lineales	3
1.1. Método de Bisección	3
1.2. Método de Newton	6
1.3. Método de los Puntos Fijos	10
2. Ecuaciones Diferenciales Ordinarias	15
2.1. Método de Euler	15
2.2. Método de Taylor de orden 2	18
2.3. Método de Heun de orden 2	20
2.4. Método de Runge-Kutta de orden 4	23
3. Extensión de Ecuaciones Diferenciales	27
3.1. Para dos ó más EDOs de 1º orden	27
3.2. Para EDOs de orden mayor a 1	27
3.3. Método de Euler	28
3.4. Método de Taylor de orden 2	29
4. Interpolación	30
4.1. Método Tradicional	30
4.2. Polinomio de Lagrange	31
4.3. Polinomio de Newton	31
4.4. Ejemplo Global	33
4.5. Límites de Interpolación Polinómica	34
4.6. Interpolación Segmentaria	34
4.7. Integración Numérica	35
4.8. Orden de precisión M	37
4.9. Error de aproximación	37
4.10. Fórmulas de cuadratura compuestas	38

Capítulo 1

Ecuaciones No Lineales

Se obtienen aproximaciones a los ceros de las funciones.

1.1. Método de Bisección

Ingredientes

- Una función.
- Un intervalo inicial.
- Una precisión solicitada o número de iteraciones.

Receta

Teorema de Bolzano: Sea f continua en $[a, b]$ tal que $f(a) \cdot f(b) < 0$. Entonces existe $c \in (a, b)$ tal que $f(c) = 0$. Si además f' tiene signo constante en el intervalo, puedo asegurar que hay un único cero en el intervalo.

- Se aplica el *Teorema de Bolzano* para encontrar un intervalo que contenga a la raíz.
- Se divide en dos y se localiza la mitad en la que se encuentra la raíz.
- Se itera.

Este método proporciona una cota para el error:

$$|Error| \leq \frac{b_n - a_n}{2} \equiv \delta$$

donde $[a_n, b_n]$ es el último intervalo (el más chico).

Si se cuenta con un numero n de iteraciones, resulta más útil escribir:

$$|Error| \leq \frac{b_0 - a_0}{2^{n+1}} \equiv \delta$$

Normalmente conviene fijar la precisión (el error máximo δ) con la que se quiere calcular el resultado y, en base a eso, calcular n :

$$n = \text{int} \left[\frac{\ln(b_0 - a_0) - \ln(\delta)}{\ln(2)} \right]$$

1.1.1. Calculadora

Haciendo las cuentas con una calculadora, los datos deben mostrarse en una tabla como la siguiente:

n	a	b	c	$f(a)$	$f(b)$	$f(c)$

Se puede incluir la cota del error mostrando que para la aproximación pedida, la cota es menor a la precisión indicada.

1.1.2. Octave

En el editor

```
function [c, err, yc]=bisecf(a,b,delta)
ya=f(a)
yb=f(b)
If ya*yb>0, break, end
max i=1 + round((log(b-a)-log(delta))/log(2));
for k=1:max i
    c=(a+b)/2
    yc=f(c);
if yc==0
        a=c;
        b=c;
else if yb*yc>0
        b=c
        yb=yc
else
        a=c;
        ya=yc;
end
    if (b-a)/2 < delta, break, end
end
c=(a+b)/2;
err=abs(b-a)/2;
yc=f(c);
```

En la *Command Window*

```
>> function y=f(x)
y=...;
end

>>[c, err, yc]=bisectf(a,b,delta)
```

siendo:

- a y b los extremos del intervalo y delta la cota del error con que se desea obtener la aproximación.
- c es la aproximación a la raíz.
- yc es f(c).
- err es la cota del error (semi amplitud del último intervalo) ($\text{err} \leq \text{delta}$).

1.1.3. Ejemplo

Obtener la raíz de $\exp(x) = \sin(x)$ más proxima a 0 en el intervalo $[-3, 2; -3, 1]$ con un error menor que $10^{-3} = \delta$.

Calculadora

- f es continua en $[-3, 2; -3, 1]$.
- $f = \exp(x) - \sin(x)$.
- $f(-3, 2) = -0,0176\dots$
- $f(-3, 1) = 0,0866\dots$
- $f(-3, 2) \cdot f(-3, 1) < 0$.
- $f' = \exp(x) - \cos(x) > 0, \forall x \in [-3, 2; -3, 1]$.

Hay un sólo cero en el intervalo $[-3, 2; -3, 1]$.

n	a	b	c	$f(a)$	$f(b)$	$f(c)$
0	-3,2	-3,1	-3,15	-	+	+
1	-3,2	-3,15	-3,175	-	+	+
2	-3,2	-3,175	-3,1875	-	+	-
3	-3,1875	-3,175	-3,18125	-	+	+
4	-3,1875	-3,18125	-3,184375	-	+	-
5	-3,184375	-3,18125	-3,1828125	-	+	+
6	-3,184375	-3,1828125	-3,18359375	-	+	-

donde

$$n = \text{int} \left[\frac{\ln(-3,1 + 3,2) - \ln(10^{-3})}{\ln(2)} \right] = 6$$

y

$$c = -3,184 \quad (\text{se redondea})$$

La cota de error es: $7,8125 \times 10^{-4}$.

Octave

```
>> function y=f(x)
y=exp(x)-sin(x);
end

>>[c, err, yc] = bisectf(-3.2, -3.1, 1e-3)
c=-3.18359375
err= 7.8124999...e-004
yc= -5.5227364047606e-004
```

Con 3 decimales correctos, la aproximación a la raíz es $c = -3,184$ y la cota de error $7,8125 \times 10^{-4}$.

1.2. Método de Newton

Para poder utilizar este método, se deben cumplir las siguientes condiciones:

1. $f \in C^2$.
2. $f(a) \cdot f(b) < 0$ (a y b valores de un determinado intervalo).
3. $f'(x) > 0$ ó $f'(x) < 0$ - monótona (*Teorema de Bolzano*). Si no se puede evaluar en el intervalo, se debe achicarlo.
4. $f(a) \cdot f''(a) > 0$ ó $f(b) \cdot f''(b) > 0$.

Ingredientes

- Una función f .
- Sus derivadas f' y f'' .
- El número de iteraciones.
- Un X_o cercano a la raíz.

Receta

El método aproxima la raíz mediante las tangentes de los puntos cercanos. Si $f \in C^2$ en $[a, b]$, $f(a) \cdot f(b) < 0$ y los signos de f' y f'' son constantes en (a, b) , entonces:

1. Existe un único cero real r en (a, b) .

$$2. \quad \boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}}$$

Esta sucesión converge a r en forma monótona si el valor inicial x_0 se elige igual al extremo del intervalo (a, b) donde f' y f'' tienen el mismo signo.

Si $E_n = |x_{n+1} - x_n|$ y se cumplen las hipótesis ya enunciadas, se verifica que:

$$\begin{array}{c} E_{n+1} = \mathcal{O}(E_n^2) \\ \downarrow \\ \text{Es del orden de} \\ \frac{E_{n+1}}{E_n^2} \xrightarrow{n \rightarrow \infty} C(cte) \end{array}$$

Esta forma de converger se denomina *Convergencia Cuadrática*.

Por ejemplo, si en el paso k se cumple que $E_k \approx 10^{-6}$, en el paso siguiente $E_{k+1} \approx 10^{-12}$.

Cuanto más alejado ponga el valor inicial de la raíz, más pasos voy a tener que hacer. Este método no posee una cota para el error pero converge mucho más rápido que el *Método de Bisección*, es más eficiente ya que hace menos iteraciones.

1.2.1. Calculadora

- Ingreso x_0 .
- Pongo $\boxed{=}$
- Pongo $\boxed{\text{ans}}$ - $\frac{f(\text{ans})}{f'(\text{ans})}$
- Pongo $\boxed{=}$ hasta que no cambie la respuesta. Esa es la aproximación.

Puedo escribir los datos en una tabla como la siguiente:

n	x_n	$f(x_n)$	$f'(x_n)$	Estimación del error

1.2.2. Octave

En el *Command Window*

```
>> x(1)=-3;
>> for k=1:4
        a=x(k);
        x(k+1)=a-(exp(a)-sin(a)/(exp(a)-cos(a)));
    end
>>x'
ans=
-3.00000
-3.18360
-3.18306
-3.18306
-3.18306
-3.18306
└
```

En el editor

```
function [Po, err, k, y]=newton(f,df,po,delta,epsilon,max1)
for k=1:max1
    p1=po-feval(f,po)/feval(df,po);
    err=abs(p1-po);
    po=p1;
    y=feval(f,po);
    if (err<delta)(abs(y)<epsilon), break, end
end
```

Entrada:

- $po \rightarrow$ aproximación inicial a la raíz de f .
- $delta \rightarrow$ tolerancia para po .
- $epsilon \rightarrow$ tolerancia para los valores de y .
- $max1 \rightarrow$ número máximo de iteraciones.

Salida:

- $po \rightarrow$ aproximación de Newton para la raíz de f .
- $err \rightarrow$ estimación del error para po .
- $k \rightarrow$ número de iteraciones.
- $y \rightarrow f(po)$.

En el *Command Window*

```
>> function y=f(x)
y=...;
end

>> function y=fp(x)
y=...;
end

>>[po,err,k,y]=newton('f','fp',po,delta,epsilon,max1)
po=
err=
k=
y=
```

1.2.3. Ejemplo

Obtener la raíz de $\exp(x) = \sin(x)$ en el intervalo $[-3, 2; -3, 1]$ con un error menor a 10^{-3} .

Calculadora

- $f = \exp(x) - \sin(x)$.
- $f \in C^2$ en $[-3, 2; -3, 1]$.
- $f(-3, 2) < 0$.
- $f(-3, 1) > 0$.
- $f(-3, 2) \cdot f(-3, 1) < 0 \rightarrow \text{Bolzano}$.
- $f' = \exp(x) - \cos(x) > 0, \forall x \in [-3, 2; -3, 1]$.
- $f'' = \exp(x) + \sin(x) > 0, \forall x \in [-3, 2; -3, 1]$.

Hay un sólo cero real r en (a, b) .

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Notese que tanto $f(-3, 1)$ como $f''(-3, 1)$ son positivos, luego, si tomo como valor inicial $x_0 = -3, 1$, la sucesión converge de forma monótona a la raíz.

n	x_n	$f(x_n)$	$f'(x_n)$	Estimación del error
0	-3, 1	0,086629864	1,044184356	0,082964148
1	-3,182964148	$1,028784 \times 10^{-4}$	1,040606893	$9,8864 \times 10^{-5}$
2	-3,183063012	$4,096 \times 10^{-10}$	1,040598701	

$\therefore c = -3,183$ y estimación del error $9,8864 \times 10^{-5} (< 10^{-3})$.

Octave

```

>>function y=f(x)
    y=exp(x)-sin(x)
end
>>function y=fp(x)
    y=exp(x)-cos(x)
end
>>[po,err,k,y]=newton('f','fp',-3.1,1e-3,1e-6,100)
po=-3.1831
err=9,8864e-005
k=2
y=4.0991e-010

```

$\therefore c = -3,183$ y estimación del error $9,8864 \times 10^{-5}$.

1.3. Método de los Puntos Fijos

Despejar x . Hay más de una manera de despejar, entonces si el error es muy grande hay que despejar distinto.

Ingredientes

- Una función f .
- Un valor inicial y un intervalo.
- Una posición solicitada o el número de iteraciones.

Receta

La función que me dan es una $f(x)$ en un intervalo $[a, b]$. De la ecuación $f(x) = 0$ se explicita una de las x obteniéndose una ecuación de la forma $g(x)$.

Ejemplos

1. $\sin(x) = 0$
 $x = \sin(x) + x$
 $\implies g(x) = \sin(x) + x$
2. $x^2 - 2x + 3 = 0$
 $x = \frac{x^2+3}{2}$
 $\implies g(x) = \frac{x^2+3}{2}$

La función $g(x)$ tiene como punto fijo la raíz de $f(x)$ en el intervalo $[a, b]$.

1.3.1. Punto fijo

Un *punto fijo* es un punto en donde la función g se intersepta con la recta $y = x$.

- $g(x)$ es continua en $[a, b]$.
- $g(x) \in [a, b], \forall x \in [a, b]$, es decir, la imagen de g está contenida en el dominio de g . (*Condición de mapeo*).
- Existe $0 < k < 1$ tal que $|g'(x)| < k, \forall x \in [a, b]$.

1. Existe un único punto fijo en el intervalo.

2. $x_{n+1} = g(x_n)$

- Si $g'(x) > 0 \implies$ converge a r en forma monótona (ESCALERA).
- Si $g'(x) < 0 \implies$ converge a r en forma alternada (TELARAÑA).
- Si $E_n = |x_{n+1} - x_n|$ y se cumplen las hipótesis ya enunciadas, se verifica que:

$$E_{n+1} = \mathcal{O}(E_n) \quad E_{n+1} \leq K E_n$$

1.3.2. Calculadora

- Ingreso x_o .
- Pongo $\boxed{=}$
- Pongo $g(ans)$.
- Pongo $\boxed{=}$ hasta que no cambie la respuesta. Ese valor sería la aproximación.

Luego puedo escribir los datos en una tabla como la siguiente:

n	x_n	Estimación del error

1.3.3. Octave

En el editor

```
function [xo, err, k]=ptofijo(xo, delta, maxl)
for k=1:maxl
    x1=g(xo);
    err=abs(x1-xo)
    if (err<delta);break, end
end
```

Entrada:

- $x_0 \rightarrow$ aproximación inicial a la raíz.
- $\delta \rightarrow$ precisión.
- $\max 1 \rightarrow$ número máximo de iteraciones.

Salida:

- $x_0 \rightarrow$ aproximación de la raíz.
- $\text{err} \rightarrow$ cota del error obtenida como el valor absoluto de la diferencia entre los últimos dos términos.
- $k \rightarrow$ número de iteraciones realizadas.

En el *Command Window*

```
>>function y=f(x)
    y=...;
end

>>[xo, err, k]=ptofijo(xo, delta, max1)
    xo=
    err=
    k=
```

1.3.4. Ejemplo

$$f(x) = x \cosh\left(\frac{10}{x}\right) - x - 6 \text{ en } [9; 10]$$

$$x = x \cosh\left(\frac{10}{x}\right) - 6$$

$$g(x) = x \cosh\left(\frac{10}{x}\right) - 6$$

$$g'(x) = \cosh\left(\frac{10}{x}\right) - \left(\frac{10}{x}\right) \sinh\left(\frac{10}{x}\right)$$

- g es continua en $[9; 10]$.
- $9 < 9,1512 = g(9) \leq g(x) \leq g(10) = 9,4305 < 10 \implies g(x) \in [9, 10], \forall x \in [9, 10]$.
- $|g'(x)| < |g'(9)| < 1, \forall x \in [9, 10]$.

$\therefore$ Existe un único punto fijo de g en $[9, 10]$.

$$x_{n+1} = g(x_n)$$

Calculadora

n	x_n
0	9
1	9,15116
2	9,18076
3	9,18714
4	9,18855
5	9,18885
6	9,18892
7	9,18894
8	9,18894
9	9,18894

$\therefore c = 9,18894$ con un error menor a 10^{-5} .

Octave

```
>>function y=g(x)
    y=x*cosh(10/x)-6;
end

>>[aprox , err , k]=ptofijo(9,1e-5,100)
    aprox=9.18894
    err=3.31706 1e-6
    k=8
```

$\therefore c = 9,18894$.

Aclaración

$e^{\frac{x}{y}} = x$ tiene dos raíces reales:

- en $[1, 2]$, $r_1 \approx 1,42961$.
- en $[8, 9]$, $r_2 \approx 8,61316$.

Dependiendo de cómo se explice la x , se puede aproximar una raíz o la otra.

$$x - e^{\frac{x}{y}} = 0$$

$$x = e^{\frac{x}{y}} \implies g(x) = e^{\frac{x}{y}}$$

Utilizando como valor inicial un número menor a 1,42961 o mayor a 1,42961 (pero hasta 8,61316) se obtiene una aproximación al primer punto fijo (1,42961). Y, utilizando un número mayor a 8,61316, la sucesión diverge, se va al infinito. Con la $g(x)$ hallada no se puede obtener una aproximación del segundo punto fijo. En este caso decimos que el punto fijo 1 es *atractor* y el 2 *repulsor*.

$$x - e^{\frac{x}{y}} = 0$$

$$\ln(x) = \frac{x}{y}$$

$$4 \ln(x) = x \implies g(x) = 4 \ln(x)$$

Utilizando como valor inicial un número mayor a 8,61316 o menor a 8,61316 se obtiene una aproximación del segundo punto fijo (8,61316). Y, utilizando un número menor a 1,42961, la sucesión diverge, se va al infinito (negativo). Con la $g(x)$ hallada no se puede obtener una aproximación del primer punto fijo. En este caso decimos que el punto fijo 2 es *atractor* y el 1 *repulsor*.

Capítulo 2

Ecuaciones Diferenciales Ordinarias

Permite resolver ecuaciones del tipo $y' = f(t, y)$ con $y(t_o) = y_o$. El resultado buscado no es un punto, si no que una función $y(t)$, de modo que mediante los siguientes métodos se obtiene una sucesión de puntos (t_k, y_k) .

2.1. Método de Euler

Ingredientes

- La función $f(t_k, y_k)$.
- Valor inicial y_o .
- Un intervalo $[a, b]$.
- El número de pasos M o el paso h .

Receta

- Se calcula h como $h = \frac{b-a}{M} \iff M = \frac{b-a}{h}$.
- Se parte del valor inicial y se usa la fórmula recursiva:

$$y_{k+1} = y_k + h \underbrace{f(t_k, y_k)}_{y'_k}$$

donde:

$$\begin{aligned} t_{k+1} &= t_k + h \text{ con } k = 0, 1, \dots, M-1. \\ t_k &= a + kh. \end{aligned}$$

Errores

Global	Local	Error Final
$e_k = y(t_k) - y_k$	$\varepsilon_{k+1} = y(t_{k+1}) - y_k - hf(t_k, y_k)$	$E = y(b) - y_m$
$ e_k = \mathcal{O}(h)$	$ \varepsilon_k = \mathcal{O}(h^2)$	$ E = \mathcal{O}(h)$

Cuanto más chico sea el valor de h (ó más grande el de M), la solución obtenida será más precisa.

2.1.1. Calculadora

- Ingreso el valor inicial.
- Pongo [=]
- Pongo [ans] + $hf(t_k, y_k)$ con $t_k = a + k * h$.

⚠ Uso el k del valor anterior al que quiero calcular.

Puedo escribir los datos en una tabla así:

k	t_k	y_k

2.1.2. Octave

En el *Command Window*

```
>>function s=f(t,y)
    s=...;
    end
>>yo
    ans=yo
>>ans+h*f(a+k*h, ans)
```

y voy cambiando el valor de k . También se puede automatizar de la siguiente manera:

```
>>y=yo, for k=0:M-1; y=y+h*f(a+k*h,y); end
```

donde los valores de M , y_o , a y h se ingresan manualmente.

En el editor

```
function [T Y]=euler(a,b,ya,M)
h=(b-a)/M
T=zeros(M+1,1);
Y=zeros(M+1,1);
T(1)=a;
y(a)=ya;
for j=1:M
    T(j+1)=T(j)+h;
    k1=h*f(t(j),y(j));
    y(j+1)=y(j)+k1;
end
```


En el *Command Window*

```
>>function s=f(t,y)
s=...;
end
>>[t,y]=euler(a,b,ya,M)
t= ...
...
y= ...
...
```

2.1.3. Ejemplo

$$\begin{cases} y'(t) = \frac{t - y(t)}{2} \\ y(0) = 1 \end{cases}$$

Con $t \in [0, 3]$ y $h = 0,5$

Calculadora

k	t_k	y_k
0	0,0	1,0000
1	0,5	0,7500
2	1,0	0,6875
3	1,5	0,7656
4	2,0	0,9492
5	2,5	1,2119
6	3,0	1,5339

donde $y(0) = 1,0000$ es el valor inicial.

Octave

```
>>function s=f(t,y)
s=(t-y)/2;
end
>>[t,y]=euler(0,3,1,6)
t= 0,00000    y=1,00000
    0,50000    0,75000
    1,00000    0,68750
    1,50000    0,76562
    2,00000    0,94922
    2,50000    1,21191
    3,00000    1,53394
```

2.2. Método de Taylor de orden 2

Ingredientes

- La función $f(t_k, y_k)$.
- Valor inicial y_o .
- Un intervalo $[a, b]$.
- El número de pasos M o el paso h .

Receta

- Se deriva $y'(t) = f(t_k, y_k) \Rightarrow y''(t) = g(t, y(t))$.
- Se calcula h como $h = \frac{b-a}{M}$.
- Se parte del valor inicial y se usa la fórmula recursiva:

$$y_{k+1} = \underbrace{y_k + hf(t_k, y_k)}_{Euler} + \frac{h^2}{2} \overbrace{g(t_k, y_k)}^{y_k''}$$

Error global

$$E_{k+1} = \mathcal{O}(h^2)$$

2.2.1. Calculadora

- Ingreso el valor inicial.
- Pongo $\boxed{=}$
- Pongo $\boxed{\text{ans}} + hf(t_k, y_k) + \frac{h^2}{2} g(\underbrace{t_k}_{a+k*h}, y_k)$

$\triangleleft$ Uso el k del valor anterior al que quiero calcular.

Puedo escribir los datos en una tabla así:

k	t_k	y_k

2.2.2. Octave

En el *Command Window*

```
>>function s=f(t,y)
    s=...;
end
>>yo
ans=yo
>>ans+h*f(a+k*h,ans)+(h**2)/2*fP(a+k*h,ans)
```

y voy cambiando el valor de k . También se puede automatizar de la siguiente manera:

```
>>y=yo, for k=0:M-1;y=y+h*f(a+k*h,y)+(h**2)/2*fP(a+k*h,y);end
```

donde los valores de M , y_o , a y h se ingresan manualmente.

2.2.3. Ejemplo

$$\begin{cases} y'(t) = \frac{t - y(t)}{2} \\ y(0) = 1 \end{cases}$$

Con $t \in [0, 3]$ y $h = 0,5$

Calculadora

$$y''(t) = \frac{1}{2}(1 - y'(t)) = \frac{1}{2}\left(1 - \frac{t - y(t)}{2}\right) = \frac{1}{2} - \frac{t}{4} + \frac{y(t)}{4}$$

k	t_k	y_k
0	0,0	1,0000
1	0,5	0,8438
2	1,0	0,8311
3	1,5	0,9305
4	2,0	1,1176
5	2,5	1,3731
6	3,0	1,6821

- $h = 0,5 = \frac{3-0}{M} \implies M = 6$
- $t_k = a + kh = kh$
- $t_{k+1} = tk + h$
- $y_{k+1} = y_k + 0,5 \cdot \left(\frac{k \cdot 0,5 - y_k}{2}\right) + \frac{0,5^2}{2} \cdot \left(\frac{1}{2} - \frac{k \cdot 0,5}{4} + \frac{y_k}{4}\right)$ (Usando el valor de k del renglón anterior)

Octave

```

>>function s=f(t,y)
    s=(t-y)/2;
end
>>function s=fp(t,y)
    s=1/2-t/4+y/4
end
>>y=1; for k=0:5,y=4+0.5*f(k*0.5,y)+0.125*fp(k*0.5,y), end
    y= 0.84375
    y= 0.83105
    y= 0.93051
    y= 1.1176
    y= 1.3731
    y= 1.6821

```

Para un mismo problema, *Taylor* es más preciso que *Euler*

2.3. Método de Heun de orden 2

Ingredientes

- Una función $f(t_k, y_k)$.
- Valor inicial y_o .
- Un intervalo $[a, b]$.
- El número de pasos M o el paso h .

Receta

- Se calcula h como $h = \frac{b-a}{M}$.
- Se calculan los incrementos:
 - $k_1 = hf(t_k, y_k)$.
 - $k_2 = hf(t_{k+1}, y_k + k_1)$.
- Se parte del valor inicial y se utiliza la fórmula:

$$y_{k+1} = y_k + \left(\frac{k_1 + k_2}{2}\right)$$

- | | |
|--------------------|---------------------|
| Error Local | Error Global |
| $\mathcal{O}(h^2)$ | $\mathcal{O}(h^2)$ |

Cuando la ecuación diferencial es lineal a coef. constantes, *Taylor* de orden 2 coincide con *Heun*

2.3.1. Calculadora

- Ingreso el valor inicial.

- Pongo

- Pongo + $\left(\frac{h \overbrace{f(t_k, y_k)}^{a+kh} + h f(t_{k+1}, y_{k+1})}{2} \right)$

⚠ Uso el k del valor anterior al que quiero calcular.

Puedo escribir los datos en una tabla así:

k	t_k	y_k

2.3.2. Octave

En el editor

```
>>function [T Y]=heun(a,b,ya,M)
    h=(b-a)/M;
    T=zeros(M+1,1);
    Y=zeros(M+1,1);
    T(1)=a;
    Y(1)=ya;
    for j=1:M
        T(j+1)=T(j)+h
        k1=h*f(T(j),y(j));
        k2=h*f(T(j+1),y(j)+k1);
        y(j+1)=Y(j)+(k1+k2)/2;
    end
```

En el *Command Window*

```
>>function s=f(t,y)
    s=...;
end
>>[t y]=heun(a,b,ya,M)
t= ... y= ...
... ..
```

2.3.3. Ejemplo

$$\begin{cases} y'(t) = t - y(t) \\ y(0) = 3 \end{cases}$$

Con $t \in [0, 1]$ y $h = 0,1$

$$\begin{aligned} y_{k+1} &= y_k + \left(\frac{k_1 + k_2}{2} \right) \\ &= y_k + \left(\frac{ht_k - hy_k + ht_k + h^2 - hy_k - h^2t_k + h^2y_k}{2} \right) \\ &= y_k + ht_k - hy_k + \frac{h^2}{2} - \frac{h^2t_k}{2} + \frac{h^2y_k}{2} \\ &= y_k \left(1 - h + \frac{h^2}{2} \right) + t_k \left(h - \frac{h^2}{2} \right) + \frac{h^2}{2} \\ &= \boxed{y_k(1 - h + \frac{h^2}{2}) + k(h^2 - \frac{h^3}{2}) + \frac{h^2}{2}} \end{aligned}$$

Calculadora

k	t_k	y_k
0	0,0	3,0000
1	0,1	2,7200
2	0,2	2,4761
3	0,3	2,2649
4	0,4	2,0832
5	0,5	1,9283
6	0,6	1,7976
7	0,7	1,6888
8	0,8	1,5999
9	0,9	1,5289
10	1,0	1,4742

Octave

```

>>function s=f(t,y)
s=t-y;
end
>>[t,y]=heun(0,1,3,10)
t= 0.0   y= 3.0000
    0.1   2,7200
    0.2   2,4761
    0.3   2,2649
    0.4   2,0832
    0.5   1,9283
    0.6   1,7976
    0.7   1,6888
    0.8   1,5999
    0.9   1,5289
    1.0   1,4742

```

2.4. Método de Runge-Kutta de orden 4Ingredientes:

- Una función $f(t_k, y_k)$.
- Valor inicial y_o .
- Un intervalo $[a, b]$.
- El número de pasos M o el paso h .

Receta

- Se calcula h como $h = \frac{b-a}{M}$.
- Se calculan los incrementos:
 - $k_1 = hf(t_k, y_k)$
 - $k_2 = hf(t_k + \frac{h}{2}, y_k + \frac{k_1}{2})$
 - $k_3 = hf(t_k + \frac{h}{2}, y_k + \frac{k_2}{2})$
 - $k_4 = hf(t_{k+1}, y_k + k_3)$
- Se parte del valor inicial y se utiliza la fórmula:

$$y_{k+1} = y_k + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

- | | |
|--------------------|---------------------|
| Error Local | Error Global |
| $\mathcal{O}(h^5)$ | $\mathcal{O}(h^4)$ |

Para un mismo problema, *Runge-Kutta* de orden 4 es más preciso que *Heun*

2.4.1. Calculadora

- Ingreso el valor inicial.

- Pongo $\boxed{=}$

- Pongo $\boxed{\text{ans}}$ + $\frac{1}{6}(hf(\overbrace{t_k}^{a+kh}, y_k) + 2hf(t_k + \frac{h}{2}, y_k + \frac{k_1}{2}) + 2hf(t_k + \frac{h}{2}, y_k + \frac{k_2}{2}) + hf(t_{k+1}, y_k + k_3))$

$\triangleup$ Uso el k del valor anterior al que quiero calcular.

Puedo escribir los datos en una tabla así:

k	t_k	y_k

Las cuentas para sacar toda la tabla son muy engorrosas.

2.4.2. Octave

En el editor

```
>>function [T Y]=rk4(a,b,ya,M)
h=(b-a)/M;
T=zeros(M+1,1);
Y=zeros(M+1,1);
T(1)=a;
Y(1)=ya;
for j=1:M
    T(j+1)=T(j)+h;
    k1=h*f(T(j),y(j));
    k2=h*f(T(j)+h/2,y(j)+k1/2);
    k3=h*f(T(j)+h/2,y(j)+k2/2);
    k4=h*f(T(j+1),yk+k3)
    y(j+1)=y(j)+1/6*(k1+2*k2+2*k3+k4)
end
```

En el *Command Window*

```
>>function s=f(t,y)
s=...;
>>[t y]=rk4(a,b,ya,M)
t= ... y= ...
... ..
```


2.4.3. Ejemplo

$$\begin{cases} y'(t) = t - y(t) \\ y(0) = 3 \end{cases}$$

Con $t \in [0, 1]$ y $h = 0,1$

Calculadora

Calcular los valores de k_1 , k_2 , k_3 y k_4 y después reemplazar en la fórmula de y_{k+1} . Es muy complicado, pero si se puede aproximar el valor de un y específico; por ejemplo, $y(0,1) \rightarrow k = 1$

$$k_1 = h(t_o - y_o) = 0,1(0 - 3) = -0,3$$

$$k_2 = h(t_o + \frac{h}{2} - (y_o + \frac{k_1}{2})) = 0,1(0 + \frac{0,1}{2} - 3 + \frac{0,3}{2}) = -0,28$$

$$k_3 = h(t_o + \frac{h}{2} - (y_o + \frac{k_2}{2})) = 0,1(0 + \frac{0,1}{2} - 3 + \frac{0,28}{2}) = -0,281$$

$$k_4 = h(t_o + h - (y_o + k_3)) = 0,1(0 + 0,1 - 3 + 0,281) = -0,2819$$

$$y(0,1) = y_o + \left(\frac{k_1 + 2k_2 + 2k_3 + k_4}{6} \right)$$

$$\boxed{y(0,1) = 2,71935} \rightarrow \text{mejor aproximación que Heun}$$

Octave

Ahora sí puedo obtener todos los valores de la tabla porque lo hace el programa, sin necesidad de hacer ninguna cuenta.

```
>>function s=f(t,y)
    s=t-y;
end

>>[t y]=rk4(0,1,3,10)
t= 0      y= 3.00000
    0.1      2.71935
    0.2      2.47492
    0.3      2.26327
    0.4      2.08128
    0.5      1.92612
    0.6      1.79525
    0.7      1.68634
    0.8      1.59732
    0.9      1.52628
    1.0      1.47152
```

Errores

- *Error Absoluto*

$$E_A = |\text{ValorAproximado} - \text{ValorExacto}|$$

- *Error Relativo*

$$E_R = \frac{\text{ErrorAbsoluto}}{\text{ValorExacto}}$$

Capítulo 3

Extensión de Ecuaciones Diferenciales

¿Cómo extender los métodos cuando queremos resolver 2 ó más ecuaciones diferenciales de 1º orden (Sistema de ecuaciones de EDOs) o ecuaciones diferenciales de orden mayor a 1? Los dos problemas se formulan de la misma manera.

3.1. Para dos ó más EDOs de 1º orden

$$\begin{cases} x_1'(t) = f_1(t, x_1(t), x_2(t)) \\ x_2'(t) = f_2(t, x_1(t), x_2(t)) \\ x_1(t_o) = x_{1o} \\ x_2(t_o) = x_{2o} \end{cases}$$

Con $t \in [t_o, t_o + T]$

$$X(t) = \begin{pmatrix} x_1(t) \\ x_2(t) \end{pmatrix} \left| F(t, x(t)) = \begin{pmatrix} f_1(t, x_1(t), x_2(t)) \\ f_2(t, x_1(t), x_2(t)) \end{pmatrix} \right| X_o = \begin{pmatrix} x_{1o} \\ x_{2o} \end{pmatrix}$$

$$\therefore \begin{cases} X'(t) = F(t, x(t)) \\ X(t_o) = X_o \end{cases}$$

3.2. Para EDOs de orden mayor a 1

$$\begin{cases} y''(t) = f(t, y(t), y'(t)) \\ y(t_o) = y_o \\ y'(t_o) = y'_o \end{cases}$$

Con $t \in [t_o, t_o + T]$

$$\begin{array}{c} x_1(t) = y(t) \\ x_2(t) = y'(t) \end{array} \left| \begin{array}{c} x_1(t_o) = y_o \\ x_2(t_o) = y'_o \end{array} \right| \begin{array}{c} x_1'(t) = y'(t) = x_2(t) \\ x_2'(t) = y''(t) = f(t, \underbrace{y(t)}_{x_1(t)}, \underbrace{y'(t)}_{x_2(t)}) \end{array}$$

$$\therefore \begin{cases} X'(t) = F(t, x(t)) \\ X(t_o) = X_o \end{cases}$$

La solución aproximada es un vector.

3.3. Método de Euler

$$\begin{cases} X_{k+1} = x_k + hF(t_k, X_k) \\ X_o \end{cases}$$

3.3.1. Dos ó más EDOs de 1º orden

$$\begin{cases} x_1'(t) = 2 + x_1(t) - x_1(t)x_2(t) \\ x_2'(t) = t - x_2(t) + x_1(t)x_2(t) \\ x_1(0) = 1 \\ x_2(0) = 2 \end{cases}$$

$$\begin{pmatrix} x_{1,k+1} \\ x_{2,k+1} \end{pmatrix} = \begin{pmatrix} nm, x_{1,k} \\ x_{2,k} \end{pmatrix} + h \begin{pmatrix} 2 + x_{1,k} - x_{1,k}x_{2,k} \\ kh - x_{2,k} + x_{1,k}x_{2,k} \end{pmatrix}$$

$\downarrow$
 $\triangle$ del renglón anterior

k	t_k	$x_{1,k}$	$x_{2,k}$
0	0	1	2
1	0,1	1,1	2
2	0,2	1,19	2,03
...	...	...	...

3.3.2. EDO de orden mayor a 1.

$$\begin{cases} x''(t) = -x(t) \\ x(0) = 1 \\ x'(0) = 0 \end{cases}$$

Con $h = 0,1$

$$x_1(t) = x(t) \mid x_2(t) = x'(t) \mid x_2'(t) = x''(t)$$

$$\therefore \begin{cases} x_1'(t) = x_2(t) \\ x_2'(t) = x_2''(t) = -x_1(t) \\ x_1(0) = 1 \\ x_2(0) = 0 \end{cases}$$

$$X_{k+1} = X_k + h \overbrace{F(t_k, X_k)}^{X'_k}$$

$$\begin{pmatrix} x_{1,k+1} \\ x_{2,k+1} \end{pmatrix} = \begin{pmatrix} x_{1,k} \\ x_{2,k} \end{pmatrix} + h \begin{pmatrix} x_{2,k} \\ -x_{1,k} \end{pmatrix}$$

k	t_k	$x_{1,k}$	$x_{2,k}$
0	0	1	0
1	0,1	1	-0,1
2	0,2	0,99	-0,2
...	...	...	...

3.4. Método de Taylor de orden 2

$$\begin{cases} X_{k+1} = X_k + h \underbrace{F(t_k, X_k)}_{X'_k} + \frac{h^2}{2} \underbrace{F'(t_k, X_k)}_{X''_k} \\ X_0 \end{cases}$$

$$\begin{cases} x''(t) = -x(t) \\ x(0) = 1 \\ x'(0) = 0 \end{cases}$$

Con $h = 0,1$

$$\begin{cases} x_1(t) = x(t) \\ x_2(t) = x'(t) \end{cases} \quad \left| \quad \begin{cases} x'_1(t) = x_2(t) \\ x'_2(t) = x''(t) = -x_1(t) \end{cases} \quad \left| \quad \begin{cases} x''_1(t) = -x_1(t) \\ x''_2(t) = -x_2(t) \end{cases}$$

$$\begin{pmatrix} x_{1,k+1} \\ x_{2,k+1} \end{pmatrix} = \begin{pmatrix} x_{1,k} \\ x_{2,k} \end{pmatrix} + h \begin{pmatrix} x_{2,k} \\ -x_{1,k} \end{pmatrix} + \frac{h^2}{2} \begin{pmatrix} -x_{1,k} \\ -x_{2,k} \end{pmatrix}$$

k	t_k	$x_{1,k}$	$x_{2,k}$
0	0	1	0
1	0,1	0,995	-0,1
2	0,2	0,98	-0,199

Capítulo 4

Interpolación

Dada una cantidad finita de puntos, es posible hallar una curva (lo más simple posible) que pase por esos puntos y estimar un valor que no conocemos mediante la función de esa curva.

- Si el valor se encuentra entre los puntos medidos $\implies$ INTERPOLAR
- Si el valor se encuentra fuera de ese rango $\implies$ EXTRAPOLAR

En este curso utilizaremos únicamente polinomios. Si se cuenta con n puntos, existe un único polinomio de grado $n - 1$ que pasa por todos los n puntos. Si se quiere obtener una derivada, no hay más que derivar el polinomio. Lo mismo con las integrales: no importa por qué método se calcule, el polinomio interpolador será siempre el mismo.

4.1. Método Tradicional

Ingredientes

- Una serie de n puntos (x_k, y_k) a vincular.

Receta

El polinomio que los vincula tiene la forma:

$$y = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \dots + a_1x + a_0$$

Se puede armar un sistema de ecuaciones como el que sigue

$$\begin{cases} y_0 = a_{n-1}x_0^{n-1} + a_{n-2}x_0^{n-2} + \dots + a_1x_0 + a_0 \\ y_{n-1} = a_{n-1}x_{n-1}^{n-1} + a_{n-2}x_{n-1}^{n-2} + \dots + a_1x_{n-1} + a_0 \end{cases}$$

Se puede escribir de forma matricial y resolver para calcular $(a_0, \dots, a_{n-1})$.
Éste método es muy tedioso y largo.

4.2. Polinomio de Lagrange

Ingredientes

- Una serie de n puntos (x_k, y_k) a vincular.

Receta

Fórmula para tres puntos (polinomio de grado 2).

$$P_2(x) = y_0 \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} + y_1 \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} + y_2 \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)}$$

4.2.1. Ejemplo

4 puntos $\Rightarrow$ polinomio de grado 3.

x	y
0,0	1,000000
0,4	0,921061
0,8	0,696707
1,2	0,362358

$$P_3(x) = 1 \frac{(x - 0,4)(x - 0,8)(x - 1,2)}{(0 - 0,4)(0 - 0,8)(0 - 1,2)} + 0,921061 \frac{x(x - 0,8)(x - 1,2)}{0,4(0,4 - 0,8)(0,4 - 1,2)} \\ + 0,696707 \frac{x(x - 0,4)(x - 1,2)}{0,8(0,8 - 0,4)(0,8 - 1,2)} + 0,362358 \frac{x(x - 0,4)(x - 0,8)}{1,2(1,2 - 0,4)(1,2 - 0,8)}$$

4.3. Polinomio de Newton

Ingredientes

Una serie de $\underbrace{n}_{\text{tabla}}$ puntos (x_k, y_k) a vincular.

Receta

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0)(x - x_1)\dots(x - x_{n-1})$$

Para hallar los coeficientes, si escribo esta ecuación en forma matricial, la matriz es triangular, por lo que no es difícil calcularlos.

Otra manera de calcular los coeficientes es encontrar la **tabla de diferencias divididas**.

4.3.1. Ejemplo 1

x	y	$DD1$	$DD2$	$DD3$
1	-3	$\frac{1-(-3)}{3-1} = 2$	$\frac{0,5-2}{5-1} = -0,375$	$\frac{0,5-(-0,375)}{6-1} = 0,175$
3	1	$\frac{2-1}{5-3} = 0,5$	$\frac{2-0,5}{6-3} = 0,5$	
5	2	$\frac{4-2}{6-5} = 2$		
6	4			

$$P_3(x) = \underbrace{-3}_{y_0} + 2(x - 1) - 0,375(x - 1)(x - 3) + 0,175(x - 1)(x - 3)(x - 5)$$

4.3.2. Ejemplo 2

x	y	$DD1$	$DD2$	$DD3$
0,6	0,91200	-0,308	-0,2055	0,041667
0,7	0,83120	-0,3491	.0,1930	
0,8	0,84629	-0,3877		
0,9	0,80752			

Quiero aproximar el valor de $f(0,67)$.

$$P_0(0,67) = 0,91200$$

$$P_1(0,67) = 0,91200 - 0,308(0,67 - 0,6) = 0,89044$$

$$P_2(0,67) = 0,91200 - 0,308(0,67 - 0,6) - 0,2055(0,67 - 0,6)(0,67 - 0,7) = 0,8909$$

$$P_3(0,67) = 0,91200 - 0,308(0,67 - 0,6) - 0,2055(0,67 - 0,6)(0,67 - 0,7) + 0,041667(0,67 - 0,6)(0,67 - 0,7)(0,67 - 0,8) = 0,89091138$$

$\therefore$ Entre $P_2(x)$ y $P_3(x) \implies \boxed{f(0,67) \approx 0,8909}$ con un error menor a 10^{-4} .

Si la tabla corresponde a las evaluaciones de un polinomio, entonces la tabla de diferencias divididas presenta ceros de determinada columna en adelante.

x	y	$DD1$	$DD2$	$DD3$	$DD4$	$DD5$	$DD6$
-1	1	$\frac{(-5)-(-1)}{(1)-(-1)} = -3$	$\frac{(-9)-(-3)}{(2)-(-1)} = -2$	$\frac{(-2)-(-2)}{(3)-(-1)} = 0$	0	0	0
1	-5	-9	-2	0	0	0	
2	-14	-13	-2	0	0		
3	-27	-17	-2	0			
4	-44	-21	-2				
5	-65	-29					
8	-152						

Usando el polinomio interpolador de Newton se puede aproximar el cero de una función a partir de una tabla de valores de algún intervalo donde la función sea monótona. La interpolación se realiza en la función inversa.

x	y
0,6	0,91200
0,7	0,88120
0,8	0,84629
0,9	0,80752

Quiero aproximar el valor de x para que $J_o(x) = 0,86$.

$J_o(x) - 0,86 = 0 \implies f(x) = J_o(x) - 0,86$ donde el valor de x que busco aproximar es un cero de f .

Permutamos las columnas:

0,91200	0,6	-3,2468	-5,8180	-18,6371
0,88120	0,7	-2,8645	-3,8708	
0,84629	0,8	-2,5793		
0,80752	0,9			

$$P_0(0,86) = 0,6$$

$$P_1(0,86) = 0,6 - 3,2468(0,86 - 0,91200) = 0,7688$$

$$P_2(0,86) = 0,6 - 3,2468(0,86 - 0,91200) - 5,8180(0,86 - 0,91200)(0,86 - 0,88120) = 0,7624$$

$\therefore$ Entre $P_1(x)$ y $P_2(x) \implies \boxed{x \approx 0,76}$ con un error menor a 10^{-2} .

Como sé que 0,86 está entre 0,84629 y 0,88120, puedo usar el siguiente polinomio:

$$P_1(0,86) = 0,7 - 2,8645(0,86 - 0,88120) = \boxed{0,76}$$

Da el mismo resultado, pero se hacen menos cuentas.

4.4. Ejemplo Global

Utilizando los tres métodos.

x_k	y_k
-1	y_{-1}
0	y_0
1	y_1

4.4.1. Polinomio

3 puntos $\implies$ polinomio de orden 2

Tradicional

$$P_2(x) = a_2x^2 + a_1x + a_0$$

Lagrange

$$P_2(x) = y_{-1} \frac{(x-1)x}{2} + y_0 \frac{(x+1)(x-1)}{-1} + y_1 \frac{(x+1)x}{2}$$

Newton

$$P_2(x) = c_0 + c_1(x+1) + c_2(x+1)x$$

4.4.2. Coeficientes

Tradicional

$$\begin{pmatrix} 1 & -1 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} a_2 \\ a_1 \\ a_0 \end{pmatrix} = \begin{pmatrix} y_{-1} \\ y_0 \\ y_1 \end{pmatrix}$$

donde resultan:

- $\boxed{a_0 = y_0}$
- $\boxed{a_2 = \frac{y_1 + y_{-1} - 2y_0}{2}}$
- $\boxed{a_1 = \frac{y_1 - y_{-1}}{2}}$

Newton

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 2 & 2 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} y_{-1} \\ y_0 \\ y_1 \end{pmatrix}$$

donde resultan:

- $c_0 = y_{-1}$
- $c_1 = y_0 - y_{-1}$
- $c_2 = \frac{y_1 - 2y_0 + y_{-1}}{2}$

También se puede plantear la tabla de diferencias divididas.

4.5. Límites de Interpolación Polinómica

- Problema de la extrapolación: puede pasar que el polinomio se aleje de la tendencia de los datos (conviene no extrapolar).
- Un problema puede aparecer cuando hay demasiados puntos. En ese caso, la curva puede oscilar mucho $\implies$ debo cambiar de método (Ver *Interpolación Segmentaria*).

4.6. Interpolación Segmentaria

- $N : 1 \text{ ó } 3$
- $P_{N,k}$ un polinomio exacto de grado N , $k = 0, 1, \dots, n-1$

$$\Phi(x) = \begin{cases} P_{N,0}(x) & x_0 \leq x \leq x_1 \\ P_{N,1}(x) & x_1 \leq x \leq x_2 \\ \vdots & \\ P_{N,n-1}(x) & x_{n-1} \leq x \leq x_n \end{cases}$$

$$\boxed{N = 1 \text{ LINEAL}}$$

Lineal en cada tramo
 Φ de clase C^0

(La derivada primera no es continua)

$$\boxed{N = 3 \text{ SPLINES}}$$

Cúbica en cada tramo
 Φ de clase C^2

(Las derivadas primera y segunda son continuas)

$\boxed{\text{La de Splines es mayor que el polinomio interpolador y que } N = 1}$

4.6.1. Ejemplo de Splines

$$\begin{cases} P(x) = a_3x^3 + a_2x^2 + a_1x + a_0 & \text{en } [1; 2] \\ Q(x) = b_3x^3 + b_2x^2 + b_1x + b_0 & \text{en } [2; 3] \end{cases}$$

ambas P y Q cúbicas por tramos.

Datos

$$\begin{aligned} (1) P(1) &= 4 & (2) P''(1) &= 0 & (5) P(2) &= 3 \\ (3) Q(3) &= 6 & (4) Q''(3) &= 0 & (6) Q(2) &= 3 \end{aligned}$$

Nótese que las igualdades en (5) y (6) garantizan la continuidad. Además, *Splines* me garantiza continuidad de las derivadas de 1er. y 2do. orden en $[1; 3]$. Luego

$$P'(2) = Q'(2) \quad (7)$$

$$P''(2) = Q''(2) \quad (8)$$

Planteo el sistema (8 ecuaciones, 8 incógnitas).

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 6 & 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 27 & 9 & 3 & 1 \\ 0 & 0 & 0 & 0 & 18 & 2 & 0 & 0 \\ 8 & 4 & 2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 8 & 4 & 2 & 1 \\ 12 & 4 & 1 & 0 & -12 & -4 & -1 & 0 \\ 12 & 2 & 0 & 0 & -12 & -2 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} a_3 \\ a_2 \\ a_1 \\ a_0 \\ b_3 \\ b_2 \\ b_1 \\ b_0 \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \\ 6 \\ 0 \\ 3 \\ 3 \\ 0 \\ 0 \end{pmatrix} \quad \begin{matrix} (1) \\ (2) \\ (3) \\ (4) \\ (5) \\ (6) \\ (7) \\ (8) \end{matrix}$$

4.7. Integración Numérica

$$I = \int_a^b f(x) dx = Q[f] + E[f]$$

donde Q es la *Fórmula de Cuadratura* y E el *Error de Aproximación*.

4.7.1. Fórmulas de cuadratura simple

Se usan polinomios.

Si $x_{k+1} - x_k = h > 0 \rightarrow$ nodos igualmente espaciados $\rightarrow$ *Newton-Cotes* (ejemplo: Trapecio, *Simpson*, *Boole*, etc.)

N = número de puntos que se toman.

$$\boxed{N = 1}$$

Reglas del rectángulo

- *Izquierda*: $I \approx (b - a) \cdot f(a) \quad (M = 0)$
- *Derecha*: $I \approx (b - a) \cdot f(b) \quad (M = 0)$.
- *Punto Medio* o *Regla de Gauss de un punto*: $I \approx (b - a) \cdot f\left(\frac{a+b}{2}\right) \quad (M = 1)$

$$\boxed{N = 2}$$

Trapecio clásica $\longrightarrow$ *Newton-Cotes*

$$I \approx \frac{b-a}{2} \cdot (f(a) + f(b)) \quad M = 1$$

Regla de Gauss de dos puntos

$$I \approx \frac{b-a}{2} \cdot (f(x_0) + f(x_1)) \quad M = 3$$

con

$$x_0 = \frac{a+b}{2} - \frac{1}{\sqrt{3}} \frac{b-a}{2} \quad \wedge \quad x_1 = \frac{a+b}{2} + \frac{1}{\sqrt{3}} \frac{b-a}{2}$$

$$\boxed{N = 3}$$

Simpson 1/3 $\longrightarrow$ *Newton-Cotes* ($M = 3$)

$$I \approx \frac{b-a}{6} (f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)) \quad (4.1)$$

$$I_S = \frac{1}{3} \underbrace{I_T}_{\text{trapecio}} + \frac{2}{3} \underbrace{I_{PM}}_{\text{punto medio}} \quad (4.2)$$

Regla de Gauss de tres puntos: aproxima mejor que *Simpson*.

$$I \approx \frac{b-a}{18} (5f(x_0) + 8f(x_1) + 5f(x_2)), \quad M = 5$$

con

$$\begin{aligned} x_0 &= \frac{a+b}{2} - \sqrt{\frac{3}{5}} \frac{b-a}{2} \\ x_1 &= \frac{a+b}{2} \\ x_2 &= \frac{a+b}{2} + \sqrt{\frac{3}{5}} \frac{b-a}{2} \end{aligned}$$

$$\boxed{N = 4}$$

Simpson 3/8 $\longrightarrow$ *Newton-Cotes* ($M = 3$)

$$I \approx \frac{b-a}{8} (f(\overbrace{a}^{k=0}) + 3f(\underbrace{x_1}_{k=1}) + 3f(\overbrace{x_2}^{k=2}) + f(\underbrace{b}_{k=3}))$$

con $h = \frac{b-a}{3}$ (se divide el intervalo en tres partes iguales) y $x_k = a + kh$, con $k = 0, 1, 2, 3$. Ésta fórmula no se usa mucho.

$$\boxed{N = 5}$$

Boole $\longrightarrow$ *Newton-Cotes* ($M = 5$)

$$I \approx \frac{b-a}{90} (7f(\overbrace{a}^{k=0}) + 32f(\underbrace{x_1}_{k=1}) + 12f(\overbrace{x_2}^{k=2}) + 32f(\underbrace{x_3}_{k=3}) + 7f(\overbrace{b}^{k=4}))$$

con $h = \frac{b-a}{4}$ (se divide el intervalo en cuatro partes iguales) y $x_k = a + kh$, con $k = 0, 1, 2, 3, 4$.

4.8. Orden de precisión M

Miro hasta qué grado de polinomio la fórmula es exacta (la integral coincide con el valor que approximo con algún método).

El máximo grado de polinomio para el cual $E(f) = 0$, ese es el orden de precisión M .

- Para los métodos *Newton-Cotes*
 - $M = N$ si N es impar.
 - $M = N - 1$ si N es par.
- Para las reglas de cuadratura de *Gauss*
 - $M = 2N - 1$ (M es más grande que en las reglas de *Newton-Cotes*)

4.9. Error de aproximación

$$E[f] = Q_C(b-a)^{M+2} f^{(M+1)}(\varepsilon), \quad \varepsilon \in (a; b)$$

Por ejemplo: *Regla del Punto Medio*

$$\begin{aligned} \int_a^b f(x) dx &\approx (b-a) f\left(\frac{a+b}{2}\right) \\ \int_a^b 1 dx &= x \Big|_a^b = b-a = (b-a) \underbrace{f\left(\frac{a+b}{2}\right)}_{=1} \\ \int_a^b x dx &= \frac{x^2}{2} \Big|_a^b = \frac{b^2}{2} - \frac{a^2}{2} = (b-a) \left(\frac{a+b}{2}\right) \\ \int_a^b x^2 dx &= \frac{x^3}{3} \Big|_a^b = \frac{b^3}{3} - \frac{a^3}{3} \neq (b-a) \left(\frac{a+b}{2}\right)^2 \end{aligned}$$

$\therefore M = 1$ (máximo grado del polinomio para el cual la fórmula es exacta).

$$\begin{aligned}
E[x^2] &= \frac{b^3}{3} - \frac{a^3}{3} - (b-a) \left(\frac{b+a}{2} \right)^2 = \frac{b^3 - a^3}{3} - \frac{(b-a)(b+a)^2}{4} \\
&= \frac{b^3}{3} - \frac{a^3}{3} - \frac{(b-a)}{4} (b^2 + 2ab + a^2) = \\
&= \frac{b-a}{12} (4(a^2 + ab + b^2) - 3(a^2 + b^2 + 2ab)) \\
&= \frac{b-a}{12} (a^2 - 2ab + b^2) = \frac{(b-a)^3}{12}
\end{aligned}$$

$$\begin{aligned}
&\Rightarrow \frac{\cancel{(b-a)}^3}{12} = Q_C \cancel{(b-a)}^3 \underbrace{f''(\varepsilon)}_{=2, \forall \varepsilon} \\
\therefore Q_C &= \frac{1}{24} \Rightarrow E[f] = \frac{1}{24} (b-a)^3 f''(\varepsilon)
\end{aligned}$$

4.10. Fórmulas de cuadratura compuestas

¿CÓMO SE PUEDE MEJORAR LA APROXIMACIÓN?
AUMENTANDO EL NÚMERO DE PUNTOS

La idea de las fórmulas de cuadratura compuestas es aplicar una regla simple en cada tramo entre puntos consecutivos.

4.10.1. Regla del rectángulo compuesta

- *Izquierda:* $I \approx \sum_{k=0}^{n-1} f(x_k)(x_{k+1} - x_k)$
- *Derecha:* $I \approx \sum_{k=0}^{n-1} f(x_{k+1})(x_{k+1} - x_k)$
- *Punto medio:* $I \approx \sum_{k=0}^{n-1} \underbrace{(x_{k+1} - x_k)}_{=h} f\left(\frac{x_{k+1} + x_k}{2}\right)$

4.10.2. Simpsons

(n par)

$$\begin{aligned}
I \approx \frac{2h}{6} & (f(a) + 4f(x_1) + f(x_2) + f(x_2) + 4f(x_3) + f(x_4) + f(x_4) + \\
& + 4f(x_5) + f(x_6) + \dots + f(x_{n-2}) + 4f(x_{n-1}) + f(b))
\end{aligned}$$

con $x_{k+1} - x_k = h = \frac{b-a}{n}$

$$I \approx \frac{h}{3}(I + 2P + 4IM)$$

donde

1. $E = f(a) + f(b)$
2. $P = f(x_2) + f(x_4) + \dots + f(x_{n-2})$
3. $IM = f(x_1) + f(x_3) + \dots + f(x_{n-1})$

$$|I - S_n[f]| = \frac{b-a}{180} h^4 |f^{iv}(\varepsilon)| \quad \varepsilon \in (a; b)$$

4.10.3. Ejemplo

Aproximar $\int_0^1 t^2 e^{-t^2} dt$ con error menor que 10^{-3} , sabiendo que $|f^{(4)}(x)|$ en $[0; 1]$ es 24.

No me dice el número de subintervalos $\Rightarrow$ lo averiguo.

$$\begin{aligned} |I - S_n[f]| &= \frac{b-a}{180} h^4 |f^{iv}(\varepsilon)| \quad \varepsilon \in (a; b) \\ &= \frac{1}{180} \frac{1}{h^4} |f^{iv}(\varepsilon)| \leq \frac{1}{180} \frac{1}{n^4} 24 < 10^{-3} \end{aligned}$$

$$\frac{400}{3} < n^4$$

$$4 \leq n \quad (par)$$

$$h = \frac{b-a}{4} = 0,25$$

x_k	$f(x_k)$
0	0
0,25	0,0587
0,5	0,1947
0,75	0,3205

donde

$$E = f(a) + f(b) = 0,3679$$

$$P = 0,1947$$

$$IM = 0,3792$$

$$I \approx \frac{h}{3}(I + 2P + 4IM) = 0,190$$

$$I \approx \sum_{k=0}^{n-1} \frac{x_{k+1} - x_k}{2} (f(x_{k+1}) + f(x_k))$$

Si $x_{k+1} - x_k = h = \frac{b-a}{n} \rightarrow$ puntos equiespaciados y n : cantidad de subintervalos en que divido.

$$I \approx \frac{h}{2} \sum_{k=0}^{n-1} (f(x_{k+1}) + f(x_k)) = \frac{h}{2} (\underbrace{f(a) + f(b)}_E) + 2 \sum_{k=0}^{n-1} \underbrace{f(x_k)}_{IM}$$

$$\boxed{I \approx T_n[f] = \frac{h}{2}(E + 2IM)}$$

$$\boxed{|I - T_n[f]| = E[f] = \frac{b-a}{12} h^2 |f''(\varepsilon)|} \quad \varepsilon \in (a, b)$$

4.10.4. Ejemplo

$$\int_2^4 \ln(x+2) dx$$

Si quiero un error menor que 10^{-3} , ¿cuántos puntos voy a necesitar tomar?

$$|I - T_n[f]| = \frac{b-a}{12} h^2 |f''(\varepsilon)| = \frac{2}{12} \left(\frac{2}{n}\right)^2 \frac{1}{(\varepsilon+2)^2} \leq \frac{8}{12n^2} \frac{1}{16} = \frac{1}{24n^2}$$



el valor más grande que toma ($\varepsilon = 2$)

$$\frac{1}{24n^2} < 10^{-3} \implies \sqrt{\frac{100}{24}} < n \implies \boxed{7 \leq n} \longrightarrow \text{voy a necesitar 7 o más puntos.}$$

Si tengo $n = 4$, ¿cuál es la precisión?

$$|I - T_n[f]| \leq \frac{1}{24n^2} = \frac{1}{24 \cdot 4^2} < \boxed{10^{-2}}$$